Now, to reduce the noise, we are only interested in a few methods, specifically the functions that use a heap, in particular `_frequency_table_to_heap`, which takes the dictionary with the frequency (or number of occurrences) of each character in the input text, and creates a heap with one entry per character, and `_heap_to_tree`, which in turn takes the heap created by the former function, and uses it to create the Huffman encoding tree.

We also want to track down the calls to the heap methods used: `_heapify`, `top` and `insert`. So, instead of just printing those stats, we can read them as a dictionary, and filter the entries corresponding to those five functions.

To have meaningful, reliable results, we also need to profile several calls to the method `huffman.create_encoding`, and so processing the stats and saving the result to a CSV file seems the best choice anyway.

To see the profiling in action, check out our example on GitHub. The example profiles several calls to the method creating a Huffman encoding over an ensemble of large text files[26] and a bitmap image. The bitmap needs to be duly pre-processed in order to make it processable as text – the details of this pre-processing are not particularly interesting, but for the curious reader: we encode image bytes in base64 to have valid characters.

2.10.2 Interpreting Results

Now that we stored the results of profiling in a CSV file, we *just* need to interpret them to understand what's the best